Practical 3: File Handling and OS Module Operations

Aim

To study and implement file handling and OS module operations such as creating directories, reading/writing files, renaming, and deleting files in Python.

Theory

File handling in Python allows programs to create, read, write, append to, and manipulate files stored on persistent storage using the built-in open() function, which supports modes such as 'w' (write, truncates existing content), 'r' (read), 'a' (append), and 'x' (exclusive creation), along with binary variants like 'wb'/'rb'. Using a 'with' statement (context manager) ensures the file is automatically and safely closed even if an exception occurs, which is why it is preferred over manual open()/close() calls. The os module provides a cross-platform interface to interact with the underlying operating system: os.makedirs() creates a directory (and any missing intermediate directories, with exist_ok=True avoiding an error if it already exists), os.listdir() lists the names of entries in a directory, os.rename() renames or moves a file, os.remove() deletes a file, and os.rmdir() deletes an empty directory. Related modules such as os.path (for path manipulation and existence checks) and shutil (for recursive copy/move/delete of non-empty directories) are commonly used alongside the os module for complete file-system automation.

Algorithm

1. Import the os module.
2. Create a directory using os.makedirs().
3. Open a file in write mode and write text into it, then close it.
4. Open the same file in read mode and display its content.
5. List all files in the directory using os.listdir().
6. Rename the file using os.rename() and list the directory again.
7. Delete the file using os.remove() and delete the directory using os.rmdir().

Program

```
import os
os.makedirs("demo_folder", exist_ok=True)

with open("demo_folder/sample.txt", "w") as f:
    f.write("Hello, File Handling!\n")
    f.write("This is line two.")

with open("demo_folder/sample.txt", "r") as f:
    print("File Content:")
    print(f.read())

print("Files in folder:", os.listdir("demo_folder"))

os.rename("demo_folder/sample.txt", "demo_folder/renamed.txt")
print("After rename:", os.listdir("demo_folder"))

os.remove("demo_folder/renamed.txt")
os.rmdir("demo_folder")
print("Cleanup done")
```

Output

```
File Content:
Hello, File Handling!
This is line two.
Files in folder: ['sample.txt']
```

```
After rename: ['renamed.txt']
Cleanup done
```

Result / Conclusion

File handling and OS module operations (create, read, rename, delete) were successfully performed and verified.

Viva Questions

Q1. What are the different file opening modes in Python's open() function?

Ans: 'r' — read (default, file must exist); 'w' — write (creates or truncates); 'a' — append (creates or adds to end); 'x' — exclusive create (fails if file exists); 'b' — binary mode suffix (e.g. 'rb', 'wb'); '+' — read+write (e.g. 'r+', 'w+'). The mode determines whether the file pointer starts at the beginning or end and whether existing content is preserved.

Q2. Why should we use the 'with' statement when working with files?

Ans: The 'with' statement uses Python's context manager protocol (__enter__ and __exit__). It guarantees the file is closed automatically when the block exits, even if an exception is raised. Without it, forgetting to call f.close() can cause resource leaks (file handles not released), data corruption (buffered data not flushed), or errors when other processes try to access the file.

Q3. What is the difference between os.makedirs() and os.mkdir()?

Ans: os.mkdir() creates only a single directory and raises FileNotFoundError if any intermediate parent directory is missing. os.makedirs() creates the full path including all necessary intermediate directories (like mkdir -p in Unix). The exist_ok=True parameter suppresses the error if the directory already exists.

Q4. What is the difference between the os module and the shutil module?

Ans: The os module provides low-level, single-item file system operations: create/rename/delete individual files and empty directories. The shutil (shell utilities) module provides higher-level operations: shutil.copy() copies a file, shutil.move() moves/renames files across filesystems, and shutil.rmtree() deletes an entire directory tree (including non-empty directories), which os.rmdir() cannot do.

Q5. What happens if you open a file in 'w' mode and the file already exists?

Ans: Opening an existing file in 'w' (write) mode truncates it immediately — all existing content is erased and the file pointer is placed at position 0. If you want to add content without erasing, use 'a' (append) mode instead. If you want to create a file only if it does not already exist, use 'x' mode.